CONCORDIA UNIVERSITY
CONTINUING EDUCATION

MOD8

Advanced Web Programming

Milestone 3

Backend Development & Final Presentation

The Heist

Secure To-Do List Application

Team

BELLEPERCHE Grégoire
RELUT-VAINQUEUR Xavier
GUELLUY Eleonore
MONTENEGRO LOUREIRO Marco-Antonio
QUERREC Thomas

Instructor: Marcos Sanches

Term: Winter 2026

Submitted: 27 April 2026

github.com/gregipoux/Concordia-Efrei-GPRB-Project

1. Introduction

• Project Objectives:

In this Milestone 3 we delivered the back-end half of the **secure To-Do list** required by the assignment, integrated it with the Vue.js front-end built during Milestone 2, and tied the whole thing together as a single full-stack application. Our goals for this milestone were to:

(1) implement a **Node.js + Express** API on top of **MySQL** that handles user authentication, three task statuses (To Do / Doing / Done) and protected session management with **JWT** tokens.

(2) replace every mock data call in the front-end with real HTTP calls against this API, and rewrite the auth flow so that the JWT issued by the back-end is what gates every protected page.

(3) apply the **secure coding** practices the brief asks for (password hashing with bcrypt, validation, rate limiting, helmet, IDOR audit, etc.) and document each decision so the grader can see why each piece is there.

(4) keep the work split fairly across the five team members, with one accountable owner per lot and code reviews shared across the team.

(5) produce this report, the demo screenshots and the 5–7 min presentation script, so that the back-end work is testable and reviewable from a single submission.

• Concepts Used:

- **Client / server architecture** with a JSON REST API as the single integration surface between the SPA and the database.
- **Vue.js 3** (Composition API) with `vue-router` for navigation and `Pinia` for state management (auth + theme).
- **Tailwind CSS 4** with class-based dark mode (CSS variables), built through Vite.
- **Node.js + Express** for the HTTP layer, with `helmet`, `cors`, `express-rate-limit` and `express-validator` for hardening.
- **Prisma ORM** for the database layer instead of the Sequelize taught in class – we explain that choice in section 2.
- **JWT (HS256)** for stateless session management, signed with a 256-bit secret stored in `.env`.
- **bcrypt** (rounds = 10) for password storage. We never store plain passwords.
- **Themed mapping** between front-end labels (`The Plan / In Progress / The Loot`) and the course-mandated statuses (`To Do / Doing / Done`). The mapping is enforced both in the Prisma enum and in a single `mappers.js` module on the front.

2. Methodology

This section explains how we organized the work, how the code is structured and the techniques we used. Section 3 then walks through the actual application surfaces with screenshots.

- **Code structure:**

The repository is a small monorepo at `the_heist/` with two siblings: `frontend/` (Vue + Vite) and `backend/` (Express + Prisma). The back-end is split by responsibility – `src/middleware/auth.js` for the JWT verification, `src/lib/prisma.js` for the database client singleton, and one file per resource under `src/routes/` (auth, missions, vehicles, intel, agents). The front-end follows the same logic: `src/services/` for the cross-cutting concerns (HTTP wrapper + enum mapper), `src/stores/` for Pinia stores (auth + theme), `src/pages/` for the routed views and `src/components/` grouped by feature (board / crew / garage / intel / modals / profile / ui). This separation made the integration between P2 and P3 noticeably easier than it would have been with everything under a single folder.

- **Team task distribution:**

We split the milestone into five lots designed to minimize blocking dependencies. Each lot has a single owner accountable for delivery; reviews and pair-programming were shared across the team.

Lot	Scope	Owner	Status
P1 Foundations + Auth	Backend bootstrap, Prisma schema, MySQL migration, seed, `/login` + `/me` + JWT middleware	BELLEPERCHE Grégoire	Done
P2 CRUD Routes	Routes for Missions / Vehicles / Intel / Agents with validation and authorization	RELUT-VAINQUEUR Xavier	Done (with security audit pass – see 4.1)
P3 Frontend Integration	Vite proxy, `services/api.js` + `services/mappers.js`, AuthStore rewrite, page integrations	GUELLUY Eleonore	Done

Lot	Scope	Owner	Status
P4 Security Hardening	Helmet + CORS + rate limiting on `/login`, IDOR audit on intel POST, timing-safe bcrypt compare, online-status sync on login/logout, role-based access control on recruitment **and on every destructive `DELETE` route (missions / vehicles / intel)**, integer coercion on validated FK fields, bcrypt upgrade to 6.x for a clean production audit, automated security test suite (8 cases, `node:test` + `supertest`)	MONTENEGRO LOUREIRO Marco-Antonio	Done
P5 Documentation, Review & Presentation	This document, code-review pass across all lots, 5–7 min presentation script and live demo on April 28	QUERREC Thomas	In progress

- **Implementations and Techniques Used:**
  - **Authentication.** `POST /api/auth/login` validates the body with `express-validator`, fetches the agent by alias, and runs `bcrypt.compare`. To block alias-enumeration via timing attacks, when the alias does not exist we still run the compare against a precomputed dummy hash so the response time is identical in both branches. On success we update `isOnline = true` in the DB, sign a 12 h JWT with `{ sub, alias, role }` and return it. The login is rate-limited to 10 attempts / 15 min via `express-rate-limit`.
  - **JWT middleware.** `requireAuth` reads the `Authorization: Bearer <token>` header, verifies the signature with `process.env.JWT_SECRET`, distinguishes `token_expired` from `invalid_token`, and re-fetches the agent from the database to attach the freshest fields to `req.agent`. Every route except `/health` and `/login` is behind this middleware.
  - **CRUD pattern.** Every resource (`missions`, `vehicles`, `intel`, `agents`) follows the same shape: validators on the route, `prisma.<model>.findMany / create / update / delete`, partial updates on PUT, and Prisma error normalization at the catch level (`P2002 -> 409 alias_taken / plate_taken`, `P2003 -> 400 invalid_assignee / invalid_driver`, `P2025 -> 404 not_found`). This consistency lets the front-end display user-friendly messages without parsing free-text errors.
  - **Front-end API service.** `services/api.js` is a small `fetch` wrapper that injects the JWT from `sessionStorage` into every request, parses JSON, and throws a structured `ApiError(status, code,`

raw) on non-2xx responses. We expose one module per resource (authApi, missionsApi, …) with uniform list / create / update / remove methods. Using the same shape everywhere meant the page code stays readable.

- **Front / back label mapping.** The front-end was built in Milestone 2 with hardcoded human-readable labels (The Plan, Godfather, On Mission). The Prisma enums use SCREAMING_SNAKE (THE_PLAN, GODFATHER, …). Rather than refactor 30+ Vue components, we added a single services/mappers.js that translates both directions. As a side benefit, if we ever swap the back-end stack, only this file changes.
- **AuthStore (Pinia).** The store persists {token, agent} in sessionStorage (deliberately not localStorage, to limit credential persistence on shared machines). On logout, the store calls POST /api/auth/logout (best-effort) so the back-end can flip isOnline = false before we drop the token locally.
- **Role-based authorization.** A second middleware requireRole(...roles) checks req.agent.role after the JWT has been verified. It gates POST /api/agents (only a GODFATHER can create or promote an account) **and every destructive DELETE route on the shared resources: DELETE /api/missions/:id, DELETE /api/vehicles/:id and DELETE /api/intel/:id**. An authenticated regular AGENT who attempts any of these gets a 403 forbidden. The principle: regular operatives can read and contribute, but only the Godfather can wipe shared state.
- **Testable application layout.** The Express app is created in src/app.js (middlewares, routes, error handler) while src/index.js only calls app.listen. This split lets test/security.test.js import the app directly through supertest without binding a port. The suite (npm test, on top of node:test) covers eight scenarios: missing bearer header, expired token, malformed token, invalid mission enum payload, invalid vehicle FK type, non-Godfather recruitment attempt, **non-Godfather DELETE attempts on missions / vehicles / intel (one assertion per resource, all returning 403 forbidden)** and numeric input sanitization. All eight pass on a clean checkout, and the production npm audit --omit=dev reports zero vulnerabilities after upgrading bcrypt to ^6.0.0.

• Database design:

The schema covers four entities – Agent, Mission, Vehicle, IntelFile – with five enums (AgentRole, AgentStatus, MissionPriority, MissionStatus, VehicleStatus). The full schema is in back-end/prisma/schema.prisma; the diagram below was generated automatically from that schema using the prisma-erd-generator package, so it is guaranteed to stay in sync with the code.

MissionStatus is the enum that carries the course-mandated three statuses. The mapping THE_PLAN <-> To Do, IN_PROGRESS <-> Doing, THE_LOOT <-> Done is documented and respected by the mapper. Tags on IntelFile are stored as a JSON-encoded TEXT string because MySQL has no native array type – the front-end mapper parses on read and stringifies on write so the UI manipulates a normal JavaScript array.

prisma/seed.js populates the database with 4 agents (common password heist2026), 7 missions, 5 vehicles and 6 intel files. The script uses upsert keyed on natural unique fields (alias, plate), so re-running it never creates duplicates – any team member can reset their environment with one command.

• Why Prisma and not Sequelize:

The course (Class 7, April 7) introduced **Sequelize** as the ORM. We deliberately chose **Prisma** instead, for three reasons: (1) the declarative schema.prisma file is a single source of truth for models, enums and migrations, easier to read in a code review than a folder of Sequelize model files; (2) the generated client is type-checked, which catches a class of bugs at compile time rather than at runtime; (3) Prisma's parameterized queries provide the same SQL-injection protection as Sequelize's prepared statements. The cost

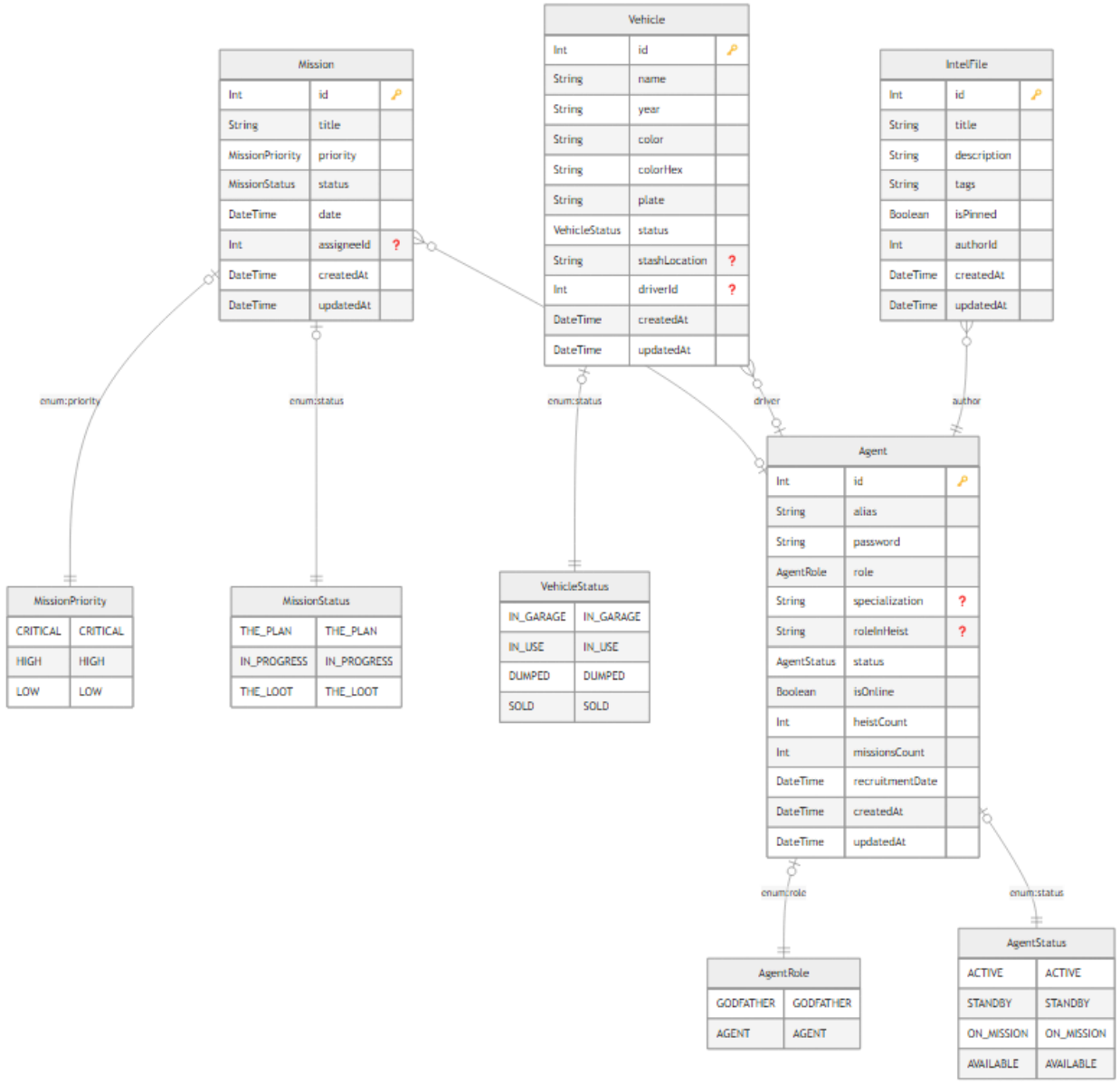

Figure 1: Database entity-relationship diagram, generated from `backend/prisma/schema.prisma`. The four entities (Agent, Mission, Vehicle, IntelFile) and their five enums are shown. Foreign-key columns (`assigneeId`, `driverId`, `authorId`) are nullable on the dependent side so that deleting an agent does not cascade-delete their missions or vehicles.

was a few extra hours of documentation reading from one team member; the MySQL database itself is unchanged. We acknowledge this is a deviation from the curriculum and we are happy to defend it during the presentation.

• **Secure coding decisions:**

The brief explicitly asks for secure coding. Below is the list of practices we applied and why each one matters in our specific context.

#	Practice	Implementation	Why it matters
1	Password hashing	`bcrypt` rounds = 10 on every stored password (seed + recruit)	Stored hashes cannot be reversed; cost = 10 is a sensible 2026 default
2	JWT signing	HS256 with a 256-bit secret in `.env` (gitignored)	Tokens cannot be forged without the secret
3	Token expiry	12 hours	Limits the exposure window if a token leaks
4	Timing-safe login	`bcrypt.compare` against a dummy hash when the alias is unknown	Blocks alias enumeration via response-time analysis
5	HTTP headers	`helmet` middleware	Defense-in-depth against XSS, clickjacking, MIME sniffing
6	CORS	Restricted to `CORS_ORIGIN` from `.env`	Prevents arbitrary websites from calling our API on behalf of the user
7	Rate limiting	`express-rate-limit` on `/login` (10 / 15 min)	Brute-force resistance
8	Input validation	`express-validator` on every route	Guarantees shape and types before they reach Prisma
9	SQL injection	Prisma's parameterized queries – we never build raw SQL	Eliminates SQL-injection vector by construction
10	**IDOR fix on `/api/intel` POST**	`authorId` is injected from the verified JWT (`req.agent.id`), never read from the body	Without this, any authenticated agent could post intel under another agent's identity
11	Sensitive storage	JWT in `sessionStorage`, not `localStorage`	Token is wiped when the browser tab closes, limiting persistence on shared machines

#	Practice	Implementation	Why it matters
12	Secret management	`.env` is in every relevant `.gitignore`; only `.env.example` is committed	Secrets never leak into the Git history
13	Body size cap	`express.json({ limit: '100kb' })`	Mitigates trivial denial-of-service via oversized requests
14	Error opacity	The global error handler returns `{ error: 'internal_error' }` for unexpected exceptions	Prevents leaking stack traces or DB internals to attackers
15	Online state sync	`/login` flips `isOnline = true`; `/logout` flips it back	Lets the application reason about active sessions
16	Role-based access control on recruitment and destructive operations	`requireRole('GODFATHER')` middleware on `POST /api/agents`, `DELETE /api/missions/:id`, `DELETE /api/vehicles/:id` and `DELETE /api/intel/:id` – only the Godfather can recruit, promote or wipe shared state	Without this, any authenticated agent could recruit (including granting themselves the Godfather role) or destroy missions, vehicles and intel files belonging to the rest of the crew
17	Integer coercion on validated body fields	`.toInt()` on `assigneeId`, `driverId`, `heistCount`, `missionsCount` after `express-validator` checks	Prisma receives numeric values rather than strings, eliminating a class of foreign-key and type errors
18	Validated optional agent fields	`status`, `isOnline`, `heistCount`, `missionsCount` are whitelisted by `express-validator` before the database insert	Stops a hostile recruit payload from injecting unknown enum values or arbitrary integer ranges
19	Up-to-date crypto dependency	`bcrypt` upgraded from `^5.1.1` to `^6.0.0`	Removes the production `npm audit --omit=dev` advisories that came from the old native dependency chain

#	Practice	Implementation	Why it matters
20	Automated security regressions	`test/security.test.js` (`node:test` + `supertest`) – 8 scenarios covering auth edge cases, input validation, recruitment authorization and destructive-route authorization	Regressions on the security surface are caught by `npm test` before the next demo

For the presentation we will live-explain practices **#4 (timing-safe), #7 (rate-limit), #10 (IDOR fix) and #16 (role-based access control on recruitment and destructive `DELETE`s)** because these are the ones a grader is most likely to probe.

3. Hands-On – Application Walkthrough

We walk through the application in the same order we will demonstrate it on April 28. Each "exercise" below is one functional surface of the final product, with the screenshot taken from the live application running locally after `npm run seed`.

Exercise 1 – Authentication and Session Security

The entry point. The user enters their **operative alias** and **retinal scan** (password). The form is wired to `auth.login(alias, password)` in the Pinia store, which calls `POST /api/auth/login` and – on success – stores `{token, agent}` in `sessionStorage` and pushes the router to `/board`. Errors are mapped by code: `invalid_credentials -> "Invalid alias or retinal scan"`, `http_429 -> "Too many attempts. Try again in 15 minutes"`, `network_error -> "Cannot reach the operations server"`. We deliberately kept the SignIn page in dark-only theme as an entry-point branding choice; the theme toggle becomes available after login.

We were initially surprised to see how much the *small* security details (rate-limit message, distinct error messages by code, button disabled while `loading`) changed the perceived quality of the form. None of these features are required by the brief, but they make the demo feel honest about what the back-end actually returns.

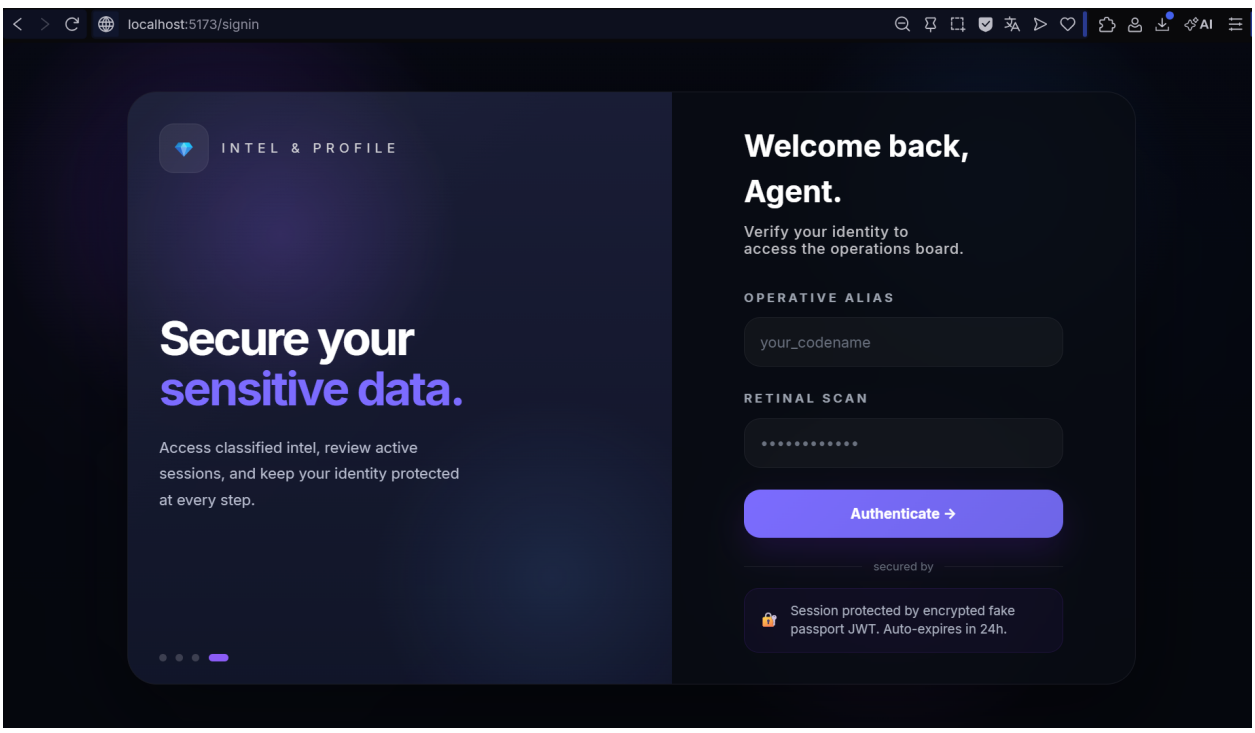

Figure 2: SignIn page (alias and retinal scan inputs); the dark gradient is kept as a branding choice and is the only screen not affected by the theme toggle.

Exercise 2 – Mission Kanban (the core To-Do)

This is the heart of the assignment: a To-Do list with three statuses. We rendered it as a Kanban board with three columns (`The Plan` / `In Progress` / `The Loot`) so that moving a task between statuses is a

visible act, which is more demonstrable than a status dropdown.

`BoardPage.vue` calls `missionsApi.list()` and `agentsApi.list()` in parallel during `onMounted`, then groups the missions by status into a computed property. Each `MissionCard` exposes Edit and Delete buttons on hover, so the default view stays clean. A mission's status is changed by opening the Edit modal and selecting a new column – we did not implement drag-and-drop in this milestone but it is on our improvements list (section 4).

Adding a mission opens `AddMissionModal`, which receives the agents list from the parent so the assignee dropdown is populated dynamically. The submit emits a `create` event up to the page, which calls `missionsApi.create(payload)` and pushes the returned (mapped) mission into the local list. A green toast confirms the operation; an error toast describes the failure (e.g. `invalid_assignee` if the chosen operative was deleted in another tab).

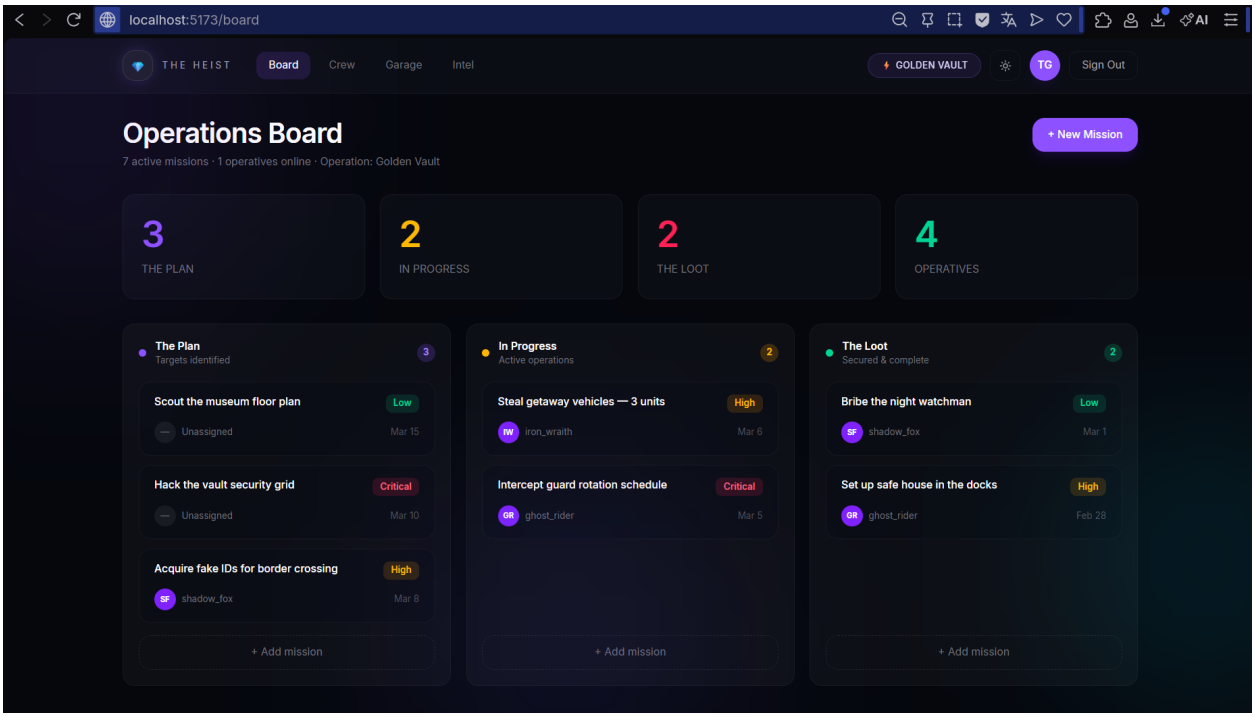

Figure 3: Board page: three Kanban columns (`The Plan`, `In Progress`, `The Loot`) populated with seeded missions; live counts in the stat cards reflect the current state of each column.

Exercise 3 – Crew Management (recruitment + online status sync)

The Crew page lists every agent in a card grid, with a green dot reflecting `isOnline`. The status was the source of one of our late bugs (section 4.3): we initially never updated `isOnline` on login, so the connected user appeared offline on their own profile. We fixed this by having `POST /api/auth/login` flip the field to `true` and a new `POST /api/auth/logout` flip it back.

The Recruit modal creates a new agent through `POST /api/agents`, with `bcrypt` hashing applied server-side and the alias unique constraint enforced (`P2002 -> 409 alias_taken`). The endpoint is gated by the `requireRole('GODFATHER')` middleware so that only a Godfather can recruit; an authenticated regular `AGENT` calling the same endpoint receives `403 forbidden`. The same gate is mounted on every destructive `DELETE` route (missions, vehicles, intel), so a regular operative can read and contribute but

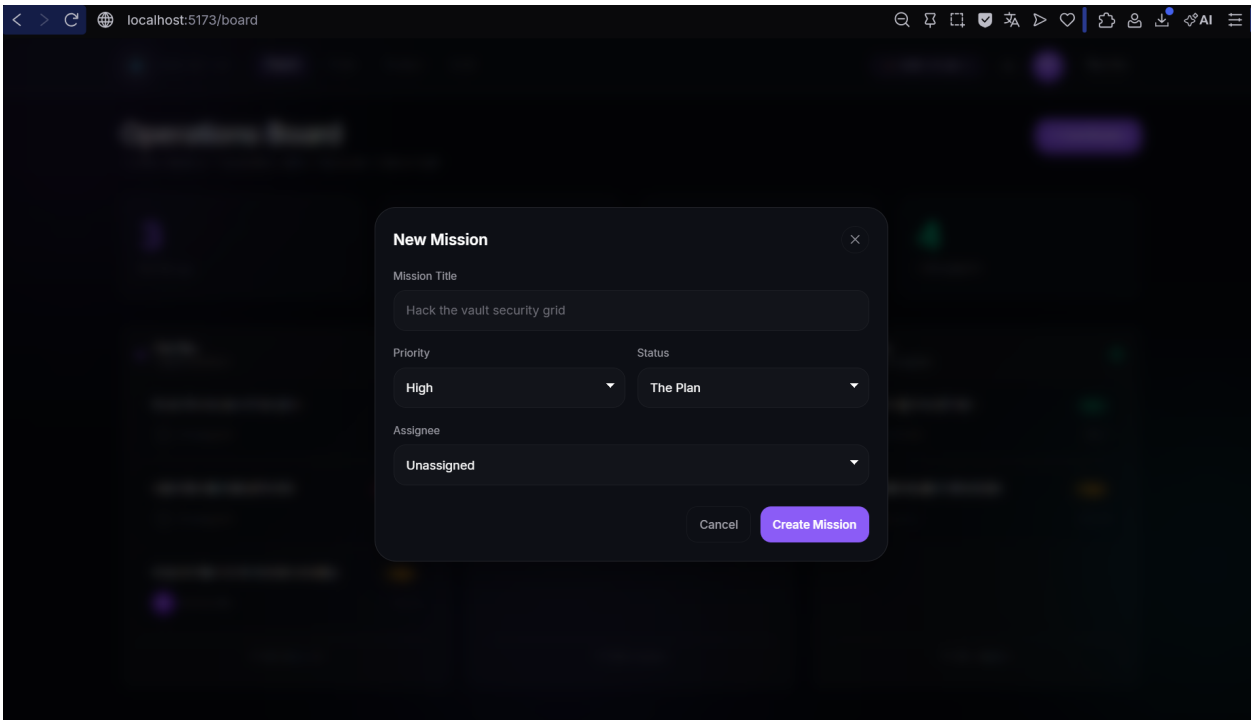

Figure 4: Add Mission modal: title, priority, status and assignee selector. The assignee dropdown is populated dynamically from `agentsApi.list()` so the available operatives match the current crew.

cannot wipe shared state – the consistency between recruitment and destruction matches the in-fiction hierarchy of the crew. The modal does not (yet) ask for a password; the back-end falls back to the seed password `heist2026` if none is provided. This is a deliberate demo simplification, called out in the recruit form copy ("Send an encrypted invite link"), and the API endpoint *does* accept a custom password if a future UI provides one.

Exercise 4 – Garage (additional CRUD with business rules)

The Garage manages stolen vehicles. It is not part of the brief's strict scope (which is "To-Do list"), but we built it as a second CRUD surface to exercise the same architecture more fully. It illustrates two interesting points:

- **Filter tabs** at the top let the user narrow the view by status (`In Garage / In Use / Dumped / Sold`). The filter is applied client-side because the list is small; for larger datasets the same code already supports a `?status=` query parameter on the API.
- **Server-side business rule** on `PUT /api/vehicles/:id`: when the new status is `DUMPED`, `driverId` is forced to `null` regardless of what the client sent. The front-end modals also disable the driver field in that state, but enforcing the invariant on the back-end means a curl call cannot bypass it. This is the right place to enforce business invariants, even if it duplicates the UI logic.

The Edit modal (figure 7) shows the color picker (`<input type="color">`) and the driver dropdown. Note that the assignee/driver references are sent as IDs (`driverId: number`), not as alias strings – the dropdown maps the displayed alias back to the agent's ID before submitting.

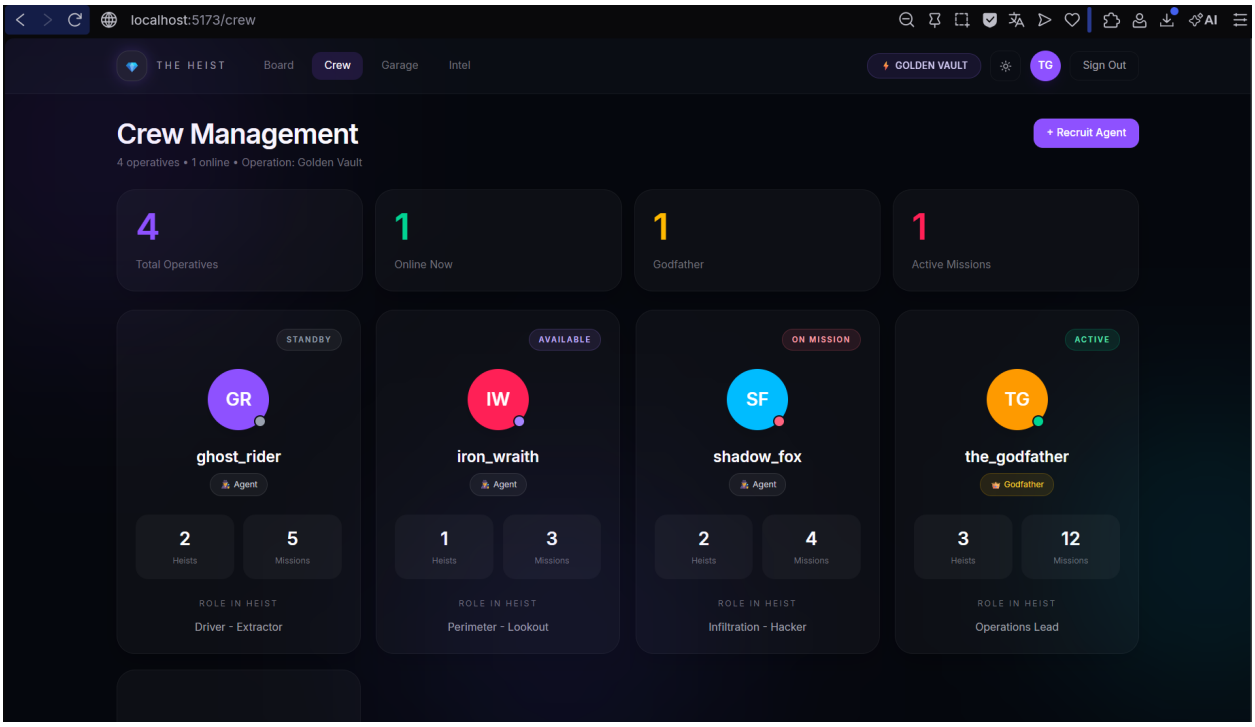

Figure 5: Crew page: agent cards with role badge, status badge and a presence dot whose colour is bound to `agent.isOnline`. The connected user (`the_godfather`) appears with a green dot once the login + logout endpoints sync the field.

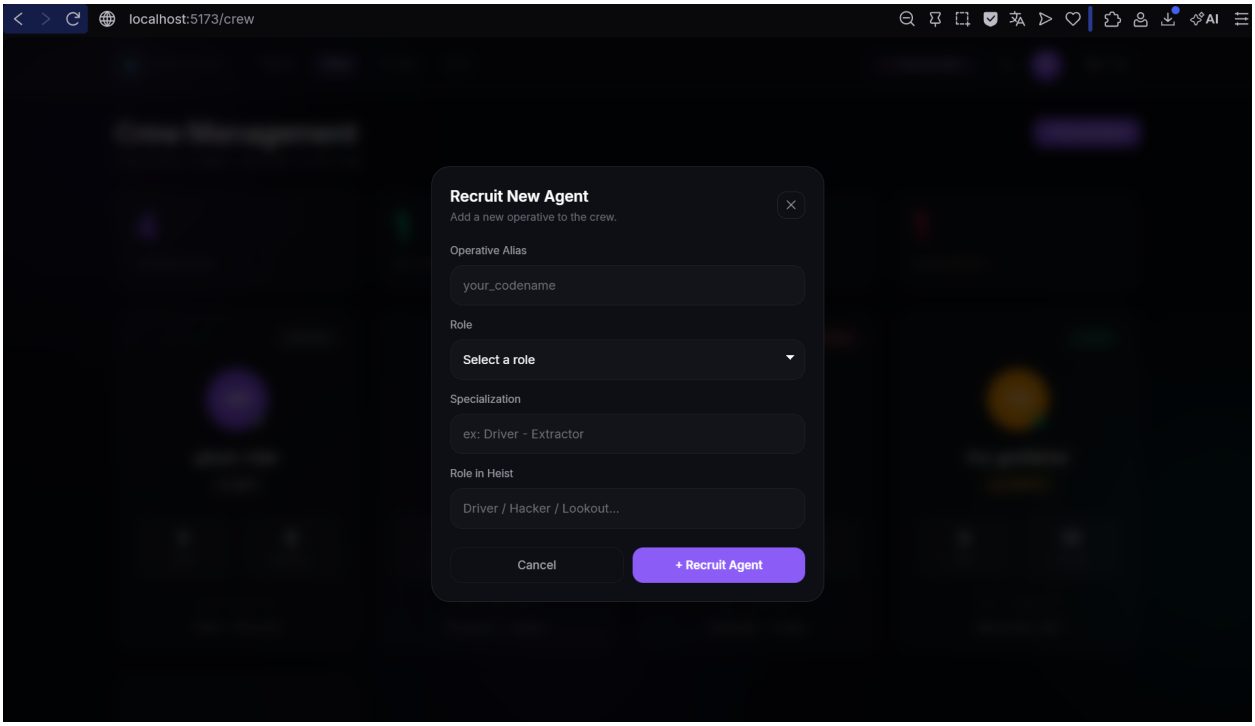

Figure 6: Recruit modal: creates a new agent through `POST /api/agents`. The password is hashed with bcrypt server-side; a duplicate alias returns HTTP 409 and is surfaced as an error toast.

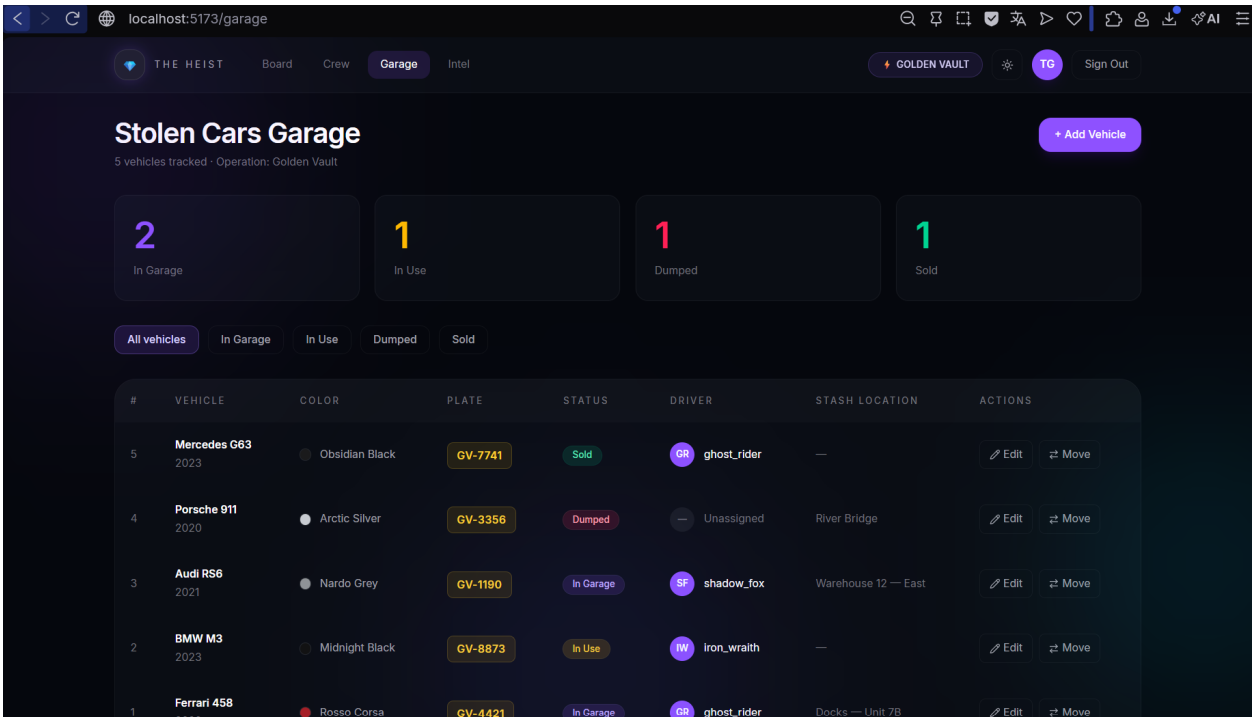

Figure 7: Garage table: client-side filter tabs (`All vehicles`, `In Garage`, `In Use`, `Dumped`, `Sold`); the same filter is also available as a `?status=` query parameter on the API for larger datasets.

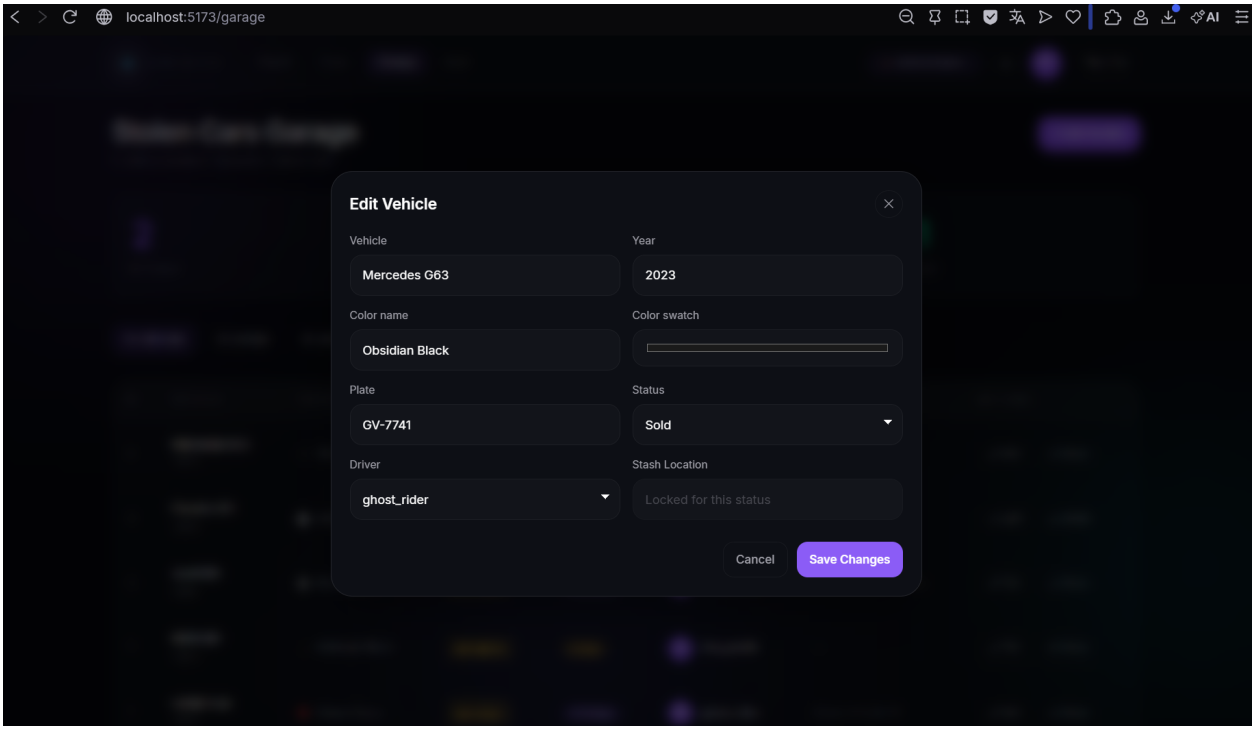

Figure 8: Edit Vehicle modal: native HTML colour picker and driver dropdown. When the status becomes Dumped, the driver field is disabled in the UI and the back-end forces `driverId` to `null` – the business rule is enforced server-side, not just in the form.

Exercise 5 – Intel (text search + pin + edit + delete)

Intel files are short notes attached to the operation. The page lists them with the pinned items on top, and the search box filters by title and description on the client (the API also supports `?search=` if needed at scale). Each card has three hover actions: edit, delete, and toggle-pin.

The pin toggle is implemented as an **optimistic update with rollback**: we flip `file.isPinned` in the local state immediately, call PUT `/api/intel/:id`, and revert the local state if the request fails. This makes the UX feel instant even on a slow network. We discovered that this same pattern would be valuable on the Kanban board if we ever ship drag-and-drop – we will reuse the rollback helper.

This page also shows our **IDOR fix in action** (the security highlight of Milestone 3). When a user creates a new intel file, the front-end never sends `authorId`. The back-end injects `authorId = req.agent.id` from the verified JWT. This means even a hand-crafted curl call cannot author intel under another agent's identity.

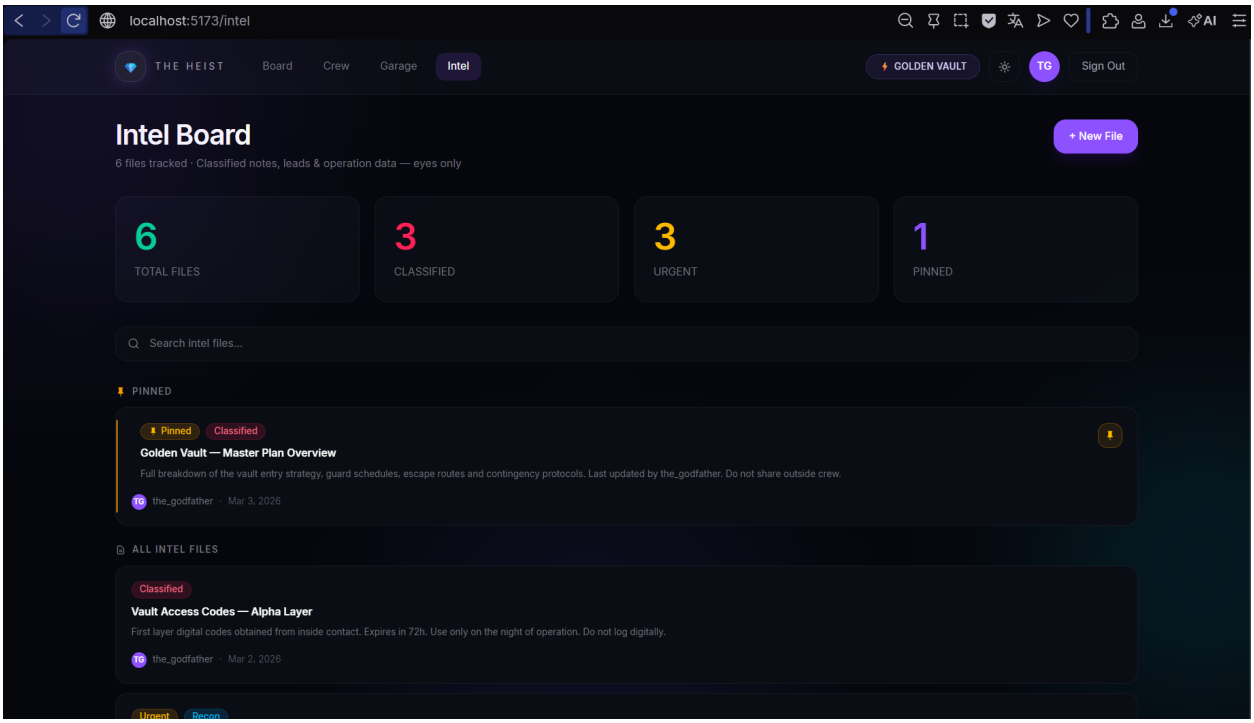

Figure 9: Intel page: pinned files appear first, followed by the rest. The search input filters by title and description on the client side. The pin toggle uses an optimistic UI update with rollback on API error.

Exercise 6 – UX Polish (toast notifications and theme toggle)

Two small features make the demo feel finished and give us material to talk about during the presentation.

Toasts (vue-sonner) fire on every create / update / delete and on auth events. Success in green, errors in red, with a close button. The toast text is mapped from the back-end error code (`alias_taken -> "This alias is already taken"`, etc.), which made it easy to keep the wording consistent across pages.

Light / dark theme toggle in the navbar (Sun / Moon icon). The theme is stored in a small `ThemeStore` (Pinia) and persisted in `localStorage`. To avoid the "flash of wrong theme" that a Vue-only solution would

cause, an inline script in `index.html` reads the stored theme and applies the class on `<html>` *before* Vue mounts. The implementation uses Tailwind 4's `@custom-variant dark` plus CSS variables for the theme tokens (`--bg-page`, `--bg-card`, `--text-primary`, etc.), so the swap is instant and consistent across every component.

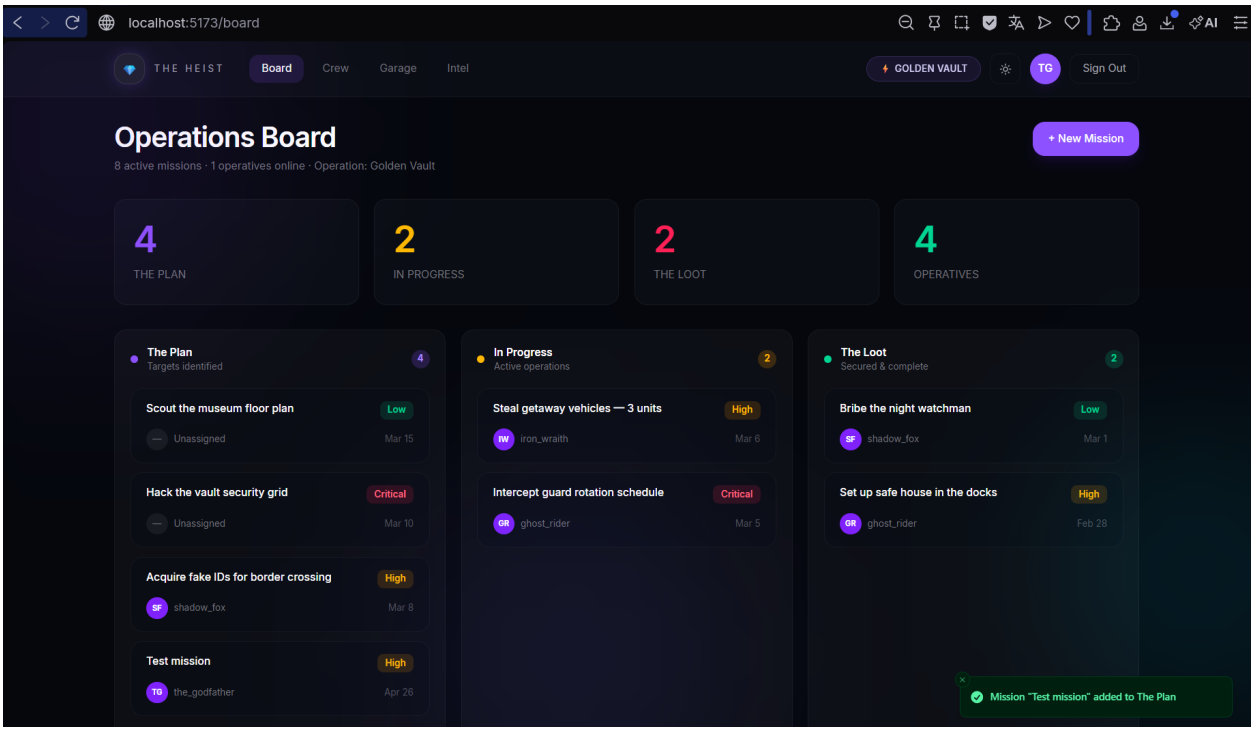

Figure 10: Toast notification (`vue-sonner`): green confirmation fired after a successful mission creation. The wording is mapped from the back-end response code so it stays consistent across all CRUD operations.

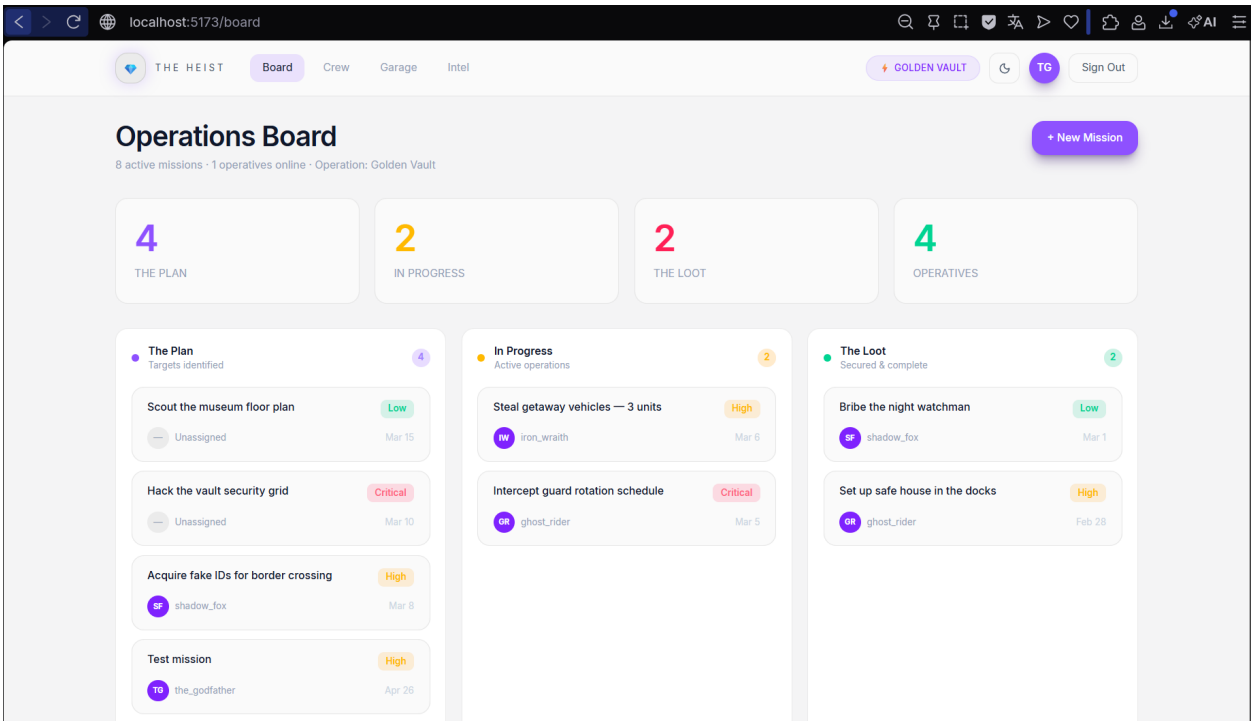

Figure 11: Light theme: the same Board page rendered in light mode after pressing the Sun/Moon toggle in the navbar. The theme is persisted in `localStorage` and re-applied before Vue mounts to avoid the flash of the wrong theme.

4. Reflections and Observations

- **Challenges Faced:**

  - **Front / back label mismatch.** Milestone 2 left us with hardcoded human-readable labels in 30+ Vue components (`The Plan`, `Godfather`, `On Mission`). The Prisma enums conventionally use `SCREAMING_SNAKE`. A naive integration would have required touching every component. We solved this with a single `services/mappers.js` that translates both directions. It was tempting to "just refactor the components", but the mapper approach turned out cleaner: components keep the labels they were built with, and the mapper became the only boundary that knows about both conventions. This is the kind of trade-off (touch one file vs touch thirty) that we will think about earlier next time.
  - **IDOR vulnerability discovered during the security audit.** Our first version of `POST /api/intel` accepted `authorId` from the request body. An authenticated agent could therefore create intel under another agent's identity. The front-end would never have done that, but the back-end should not trust the front-end. We removed `authorId` from the validator and the body, and now use `req.agent.id` from the JWT. This was the most concrete reminder that "the user is authenticated" and "the user is allowed to claim X" are two different questions.
  - **Online status not synced with the session.** Initially `Agent.isOnline` was only set by the seed script and never updated. A logged-in user appeared as offline on their own profile, which broke the green dot on the Crew page. We added `isOnline = true` on `/login` and `isOnline = false` on a new `/logout` endpoint, and made the front-end `AuthStore.logout()` async so it can call `/logout` *before* clearing the local token. Without this ordering, the logout request would have been sent without auth.
  - **Vehicle business rules.** A "Dumped" vehicle should not have a driver, and an "In Use" or "Sold" vehicle should not have a stash location. Our front-end modals disabled the relevant fields, but a curl call could have bypassed them. We added the rule to the `PUT /api/vehicles/:id` route so the back-end forces `driverId = null` when status becomes `DUMPED`, regardless of what the client sent. The principle: enforce business invariants at the data layer, not the UI.
  - **MySQL has no native array type.** `IntelFile.tags` is conceptually a list. We store it as a JSON-encoded `TEXT` column and let the mapper parse / stringify. For our scope (a handful of tags from a fixed vocabulary), this is simpler than a join table and performs well. If we ever need to query "all intel tagged Urgent" efficiently, the join table will become the right answer – but not yet.
  - **Pandoc / LaTeX rendering of this very document.** Producing this PDF with `pandoc --pdf-engine=xelatex` initially failed on missing MiKTeX packages, then on Cascadia Code not being installed system-wide. We enabled MiKTeX's auto-install (`initexmf --set-config-value=[MPM]AutoInstall=1`) and switched the monospace font to `Consolas`, which is present by default on Windows and renders the ASCII diagrams cleanly.

- **Key Takeaways:**

  - **Centralizing cross-cutting concerns pays back fast.** Two small files (`services/api.js` for HTTP / JWT injection, `services/mappers.js` for enum translation) saved us from sprinkling these concerns across every page. Once they were in place, wiring a new page took minutes, not hours.
  - **"Authenticated" is not "authorized".** The IDOR fix taught us that identity verification (JWT valid) and authorization (allowed to do this specific action with this specific data) are different layers. The back-end has to enforce both, even when the front-end already does.
  - **Optimistic UI is cheap when the rollback path is clear.** The pin toggle on Intel feels instant and the rollback on error is six lines of code. We will use the same pattern for any low-risk mutation in the future.

- **Idempotent seeds save team time.** Using `upsert` in `seed.js` means anyone on the team can reset their database with one command without worrying about duplicate-key errors. This sounds small but it removed a real friction point during P3.
- **Persisting JWT in `sessionStorage` is a small choice with a big argument.** Wiping the token when the browser tab closes is one extra sentence to say in the presentation, but it directly addresses the "shared machine" attack surface. We will reuse this default in our future projects.

• **Improvements (next steps):**

- **Drag-and-drop on the Kanban** (`vuedraggable`). We currently move missions between columns through the Edit modal; drag-and-drop is what users expect of a modern Kanban and would be a strong demo moment.
- **Per-record ownership for `PUT` / `PATCH`.** Recruitment and every destructive `DELETE` are now gated on `GODFATHER`, but updates (`PUT /api/missions/:id`, `PUT /api/vehicles/:id`, `PUT /api/intel/:id`) remain open to any authenticated agent. The natural extension is a per-record ownership model – the agent who created a record can edit it, the Godfather can edit anything – which would mirror the destructive-route gate on the modify path. This was deliberately scoped out of Milestone 3: the brief asks for "secure coding practices", which we read as authentication, validation, hardening and protection against the OWASP basics, not a full RBAC matrix on every CRUD verb.
- **Real-time updates via SSE.** A `GET /api/events` keep-alive endpoint that broadcasts mutations to all connected clients would let two browser windows react to a single click. Strong wow effect, two to three hours of work.
- **Refresh tokens** (15 min access + 7 d refresh). Standard OAuth-style design. Would reduce the token-exposure window without forcing the user to re-login often.
- **Dockerization.** A `docker-compose.yml` for MySQL + back-end + front-end so the demo runs identically on any laptop. Removes the "it works on my machine" risk for April 28.

We chose not to ship these in this milestone because the rendu deadline is two days away and the cost / benefit of any additional code change is unfavorable. The list is documented for the team's portfolio after the course.

• **Comparative Analysis (architecture choices):**

- **Prisma vs Sequelize.** Sequelize was taught in class. We picked Prisma for the declarative schema, the type-safe client and the lighter migration tooling. The trade-off is a ~3 h documentation tax on the lot owner; the upside is type-safe model access and a single source of truth that survived the entire P2 + P3 integration without drift. We are happy with the decision but would expect a 5-minute "why" justification at the demo.
- **`sessionStorage` vs `localStorage` for the JWT.** `localStorage` would persist the token across tabs and reboots, which is more convenient. We chose `sessionStorage` because the token is a credential, and a credential should not outlive the session that produced it. The cost is that the user has to log in again after closing the browser; the benefit is that a stolen device past the tab close has nothing to find.
- **Mappers (services/mappers.js) vs DTO on the back-end.** We could have made the back-end return Title-Case strings directly (e.g. respond with `"The Plan"` instead of `"THE_PLAN"`). We chose to keep the back-end on enums and translate on the client because (1) database enums catch invalid values at insert time, (2) the Title-Case is a UI concern not an API concern. If we ever ship a mobile app, the API stays clean and the new client adds its own mapping.
- **One Pinia store per concern (auth, theme) vs one global store.** Two small stores are easier to reason

about than one large one. Each store has its own API surface and its own persistence. The cost is one extra file; the benefit is that the auth flow and the theme toggle never accidentally couple.

• **Future Applications:**

The patterns we built here transfer directly to other projects we expect to work on after the course:

- **Any internal SPA + REST back-end.** The `services/api.js` + `mappers.js` + Pinia store pattern is reusable as-is. We will copy the `ApiError` shape and the JWT-injecting wrapper into our next project and only swap the resource modules.
- **Production-grade auth on a side project.** The JWT + bcrypt + helmet + rate-limit + IDOR audit checklist is a solid baseline. If we ever wire payments or sensitive PII, we would add refresh tokens, server-side session revocation and HTTPS-only cookies, but the current setup is already reasonable for an internal tool or a portfolio app.
- **Team coordination on a 5-people short-deadline project.** Splitting work into lots with a single accountable owner per lot, while sharing reviews, was effective. The bottleneck (P1 had to land before P2 and P3 could start) was identified at the start and we resourced it accordingly. We will keep this lot pattern.
- **Documentation as code.** Having this very document in the repository (`docs/MILESTONE_3.md`), built to PDF with `pandoc + xelatex`, means the doc stays in sync with the code through Git history. We will set up the same pipeline for the README of any future project.

Code, screenshots and this document are all in the repository linked at the top. Each commit is signed by its author so the per-lot contribution is verifiable from `git log`.